

Manual Técnico

PKT ERP — Plataforma de Gestión Empresarial

PROYECTO

PKT ERP

VERSIÓN

v1.0 (Mayo 2026)

ÁREA

Ingeniería y Desarrollo

CONTACTO

soporte@pkt-erp.com

1. Arquitectura del Sistema

PKT ERP está construido bajo una robusta arquitectura de **Single Page Application (SPA)** moderna, utilizando una infraestructura desacoplada de alto rendimiento basada en la nube. Este diseño garantiza una experiencia fluida, cargas de datos asíncronas inmediatas y consistencia operativa en entornos multi-inquilino (*multitenancy*).

1.1 Tecnologías Core

El núcleo técnico de la plataforma se compone de los siguientes elementos estratégicos:

- **Frontend:** React 19 compilado con **Vite**. Esta combinación permite un ciclo de renderizado optimizado, soporte nativo de componentes concurrentes y tiempos de construcción ultrarrápidos para desarrollo y producción.
- **Lenguaje:** JavaScript moderno (ES6+ / JSX) implementado bajo directivas de código limpio y tipado dinámico optimizado.
- **Estilos: TailwindCSS v4**, estructurado bajo una directiva de diseño compacta y de alta densidad que minimiza el espaciado en módulos transaccionales y centraliza la identidad a través de variables CSS personalizadas (design tokens).
- **Backend-as-a-Service (BaaS): Firebase**, aprovechando sus capacidades nativas en tiempo real:
 - **Firestore:** Base de datos documental NoSQL estructurada para comunicación en tiempo real y persistencia desconectada.
 - **Authentication:** Control de identidades federadas, sesiones persistentes y autenticación segura por tokens.
 - **Hosting:** Distribución global de activos estáticos sobre la red CDN de Google.

2. Estructura del Proyecto

La estructura del código fuente está diseñada para maximizar la modularidad y facilitar el mantenimiento continuo del sistema. Cada módulo operativo se encapsula de forma independiente para evitar colisiones de lógica.

```
/src
  /components      # Componentes visuales atómicos y compartidos
                    (Botones, Inputs, Modales)
  /context          # Proveedores de estado global reactivo
                    (AuthContext, ThemeContext)
  /hooks           # Ganchos personalizados para encapsular
                    lógica reutilizable e integraciones
  /layouts          # Estructuras de rejilla maestras para Admin,
                    Clientes y Vistas Públicas
  /modules          # Lógica de negocio y vistas encapsuladas de
                    cada módulo funcional
    /admin          # Módulos operativos exclusivos del
                    SuperAdmin
    /client          # Módulos transaccionales de los Clientes
                    (Tenants)
    /services        # Inicialización, configuración y conectores
                    con la API de Firebase
    /assets          # Recursos multimedia estáticos, logotipos e
                    iconos
```

3. Modelo de Multitenencia (Multitenancy)

PKT ERP emplea un modelo de **Aislamiento de Datos por ID de Organización** dentro de una única base de datos física de Firestore (*Single Database - Shared Schema*). Este enfoque optimiza los costes de infraestructura y simplifica el despliegue de actualizaciones globales sin comprometer la seguridad.

```
graph TD
    subgraph Firestore Database
        01[(Organización A)] --> D1[Documentos Org A]
        02[(Organización B)] --> D2[Documentos Org B]
    end
    U1[Usuario A1] -->|organizationId: A| 01
    U2[Usuario B1] -->|organizationId: B| 02

    style 01 fill:#ede9fb,stroke:#6B4FD8,stroke-width:2px
    style 02 fill:#ede9fb,stroke:#6B4FD8,stroke-width:2px
    style U1 fill:#fff,stroke:#1A1A1A,stroke-width:1px
    style U2 fill:#fff,stroke:#1A1A1A,stroke-width:1px
```

3.1 Garantías de Seguridad en el Aislamiento de Datos

El aislamiento de la información de cada cliente está protegido por tres capas de seguridad concurrentes:

1. **Filtrado en Cliente:** Todas las consultas a Firestore inyectan automáticamente el atributo `organizationId` obtenido del perfil del usuario autenticado.
2. **Validación de Reglas de Seguridad (Firestore Rules):** Reglas a nivel de servidor de base de datos que interceptan cada solicitud de lectura/escritura y validan si el `auth.token.organizationId` coincide con el del documento objetivo.

3. **Restricciones de API:** Los servicios de escritura verifican la inmutabilidad del identificador de la organización para evitar transferencias de datos accidentales.
-

4. Gestión de Estado y Autenticación

4.1 AuthContext y Seguridad de Sesión

El componente global `AuthContext` es el corazón de la gestión de estado de seguridad en la aplicación. Este proveedor automatiza los siguientes procesos críticos:

- **Persistencia de Sesión:** Integración directa con los tokens de sesión de Firebase Auth y redundancia segura mediante almacenamiento temporal en `SessionStorage`.
- **Suplantación de Identidad (Impersonation):** Capacidad exclusiva del *SuperAdmin* para simular accesos de inquilinos específicos con fines de auditoría y soporte técnico, manteniendo un rastro de auditoría transparente.
- **Control de Acceso Basado en Roles (RBAC):** Control granular que define la accesibilidad a nivel de interfaz de usuario para los roles `SuperAdmin`, `Admin` y `User`.

4.2 Esquema de Suscripción y Activación Modular

El alcance operativo de un cliente dentro de la plataforma se rige por su suscripción activa. El documento de la organización define sus límites de la siguiente manera:

ATRIBUTO	TIPO	DESCRIPCIÓN
<code>activeModules</code>	Array (Strings)	Slugs de los módulos activados (ej: <code>['crm', 'inventory', 'sales']</code>).
<code>limits.users</code>	Number	Número máximo de usuarios permitidos en la organización.
<code>limits.storage</code>	Number	Capacidad de almacenamiento de archivos adjuntos (en bytes).

5. Auditoría y Logs de Sistema

Para cumplir con estándares normativos internacionales de auditoría de TI, cada acción crítica del sistema genera un registro inmutable en la colección centralizada `audit_logs`.

Registro de Auditoría Obligatorio: No se permite ninguna operación de borrado o modificación física en configuraciones de sistema o datos financieros sin que la función de servicio invoque de manera sincrónica el método de registro de logs.

Cada documento de log contiene obligatoriamente:

- `userId` y `userName` : Identificadores únicos del operador.
 - `action` : Slug descriptivo de la acción (ej: `PAYMENT_APPROVED`).
 - `details` : Datos serializados que explican el estado previo y posterior.
 - `timestamp` : Marca temporal generada en el servidor mediante `FieldValue.serverTimestamp()` .
 - `type` : Nivel de severidad asignado para análisis visual:
 - `INFO` Información general de sesión.
 - `SUCCESS` Operaciones exitosas de negocio.
 - `WARNING` Alertas de límites de stock o suscripción.
 - `DANGER` Errores graves o accesos denegados.
-

6. Configuración y Despliegue

6.1 Variables de Entorno Requeridas

La aplicación requiere la declaración obligatoria de las credenciales de Firebase en el archivo `.env` en la raíz del entorno para su compilación exitosa:

```
# Configuración del Cliente Firebase
VITE_FIREBASE_API_KEY=AIzaSyA1...
VITE_FIREBASE_AUTH_DOMAIN=pkt-erp-prod.firebaseio.com
VITE_FIREBASE_PROJECT_ID=pkt-erp-prod
VITE_FIREBASE_STORAGE_BUCKET=pkt-erp-prod.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=8492049104
VITE_FIREBASE_APP_ID=1:8492049104:web:bf1048201
```

6.2 Comandos de Desarrollo y Producción

- **Entorno de Desarrollo:** Inicia el servidor local de Vite con recarga rápida de módulos (HMR).

```
npm run dev
```

- **Compilación para Producción:** Genera los archivos estáticos hiper-minificados en la carpeta raíz `/dist` listos para el despliegue directo a producción.

```
npm run build
```


7. Reglas de Seguridad (Firestore Rules)

El archivo `firestore.rules` ubicado en la raíz del repositorio garantiza la integridad de los datos en el servidor. El siguiente extracto describe los criterios de acceso para colecciones de organizaciones y logs:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Función auxiliar para verificar si el usuario es
    SuperAdmin
    function isSuperAdmin() {
      return request.auth != null && request.auth.token.role ==
      'superadmin';
    }

    // Función auxiliar para verificar pertenencia a la
    organización
    function belongsToOrg(orgId) {
      return request.auth != null &&
      request.auth.token.organizationId == orgId;
    }

    match /organizations/{orgId} {
      allow read, write: if isSuperAdmin() ||
      belongsToOrg(orgId);

      match /users/{userId} {
        allow read: if belongsToOrg(orgId);
        allow write: if isSuperAdmin() || (belongsToOrg(orgId)
        && request.auth.token.role == 'admin');
      }
    }

    match /audit_logs/{logId} {
      allow create: if request.auth != null;
      allow read: if isSuperAdmin() ||
      belongsToOrg(resource.data.organizationId);
    }
  }
}
```

```
        allow update, delete: if false; // Logs inmutables
    }
}
```

© 2026 PKT ERP - Equipo de Ingeniería e Infraestructura. Todos los derechos reservados.
Confidencial.